
Pointer 1

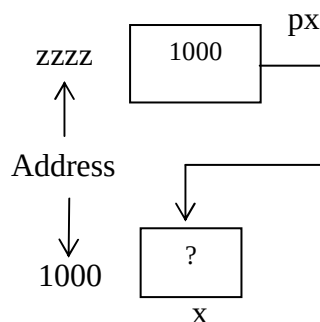
A. TUJUAN PEMBELAJARAN

1. Memahami konsep dari variabel pointer
2. Memahami cara Mengakses Isi Suatu Variabel Melalui Pointer
3. Memahami cara Mengakses dan Mengubah isi Suatu Variabel Pointer

B. DASAR TEORI

Konsep Dasar Pointer

Variabel pointer sering dikatakan sebagai variabel yang menunjuk ke obyek lain. Pada kenyataan yang sebenarnya, variabel pointer berisi alamat dari suatu obyek lain (yaitu obyek yang dikatakan ditunjuk oleh pointer). Sebagai contoh, **px** adalah variabel pointer dan **x** adalah variabel yang ditunjuk oleh **px**. Kalau **x** berada pada alamat memori (alamat awal) 1000, maka **px** akan berisi 1000. Sebagaimana diilustrasikan pada Gambar 24.1 di bawah ini



Gambar 24.1 Variabel pointer px menunjuk ke variabel x

Mendeklarasikan Variabel Pointer

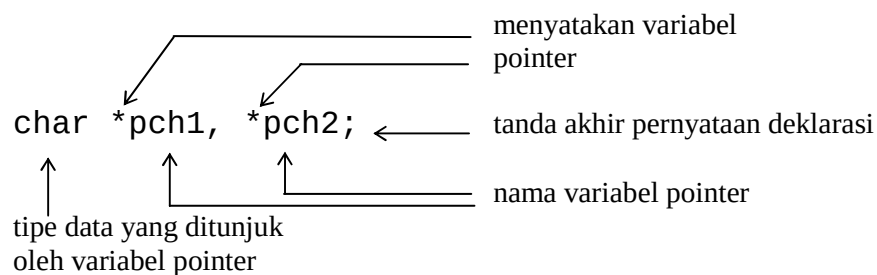
Suatu variabel pointer dideklarasikan dengan bentuk sebagai berikut :

`tipe *nama_variabel`

dengan **tipe** dapat berupa sembarang tipe yang sudah dibahas pada bab-bab sebelumnya, maupun bab-bab berikutnya. Adapun **nama_variabel** adalah nama dari variabel pointer. Sebagai contoh :

```
int *px;           //contoh 1
char *pch1, *pch2; //contoh 2
```

Contoh pertama menyatakan bahwa **px** adalah variabel pointer yang menunjuk ke suatu data bertipe *int*, sedangkan contoh kedua masing **pch1** dan **pch2** adalah variabel pointer yang menunjuk ke data bertipe *char*.



Gambar 19.2 Ilustrasi pendeklarasian variabel pointer

Mengatur Pointer agar Menunjuk ke Variabel Lain

Agar suatu pointer menunjuk ke variabel lain, mula-mula pointer harus diisi dengan alamat dari variabel yang akan ditunjuk. Untuk menyatakan alamat dari suatu variabel, operator **&** (operator alamat, bersifat *unary*) bisa dipergunakan, dengan menempatkannya di depan nama variabel. Sebagai contoh, bila **x** dideklarasikan sebagai variabel bertipe *int*, maka

`&x`

berarti “alamat dari variabel **x**”. Adapun contoh pemberian alamat **x** ke suatu variabel pointer **px** (yang dideklarasikan sebagai pointer yang menunjuk ke data bertipe *int*) yaitu :

```
px = &x;
```

Pernyataan di atas berarti bahwa **px** diberi nilai berupa alamat dari variabel **x**. Setelah pernyataan tersebut dieksekusi barulah dikatakan bahwa **px** menunjuk ke variabel **x**.

Mengakses Isi Suatu Variabel Melalui Pointer

Jika suatu variabel sudah ditunjuk oleh pointer, variabel yang ditunjuk oleh pointer tersebut dapat diakses melalui variabel itu sendiri (pengaksesan langsung) ataupun melalui pointer (pengaksesan tak langsung). Pengaksesan tak langsung dilakukan dengan menggunakan operator *indirection* (tak langsung) berupa simbol `*` (bersifat *unary*). Contoh penerapan operator `*` yaitu :

`*px`

yang menyatakan “nilai/value yang ada pada alamat/address **px**” . Sebagai contoh jika **y** bertipe *int*, maka sesudah dua pernyataan berikut

```
px = &x;  
y = *px;
```

y akan berisi nilai yang sama dengan nilai **x**.

Kedua pernyataan di atas

```
px = &x;  
y = *px;
```

sebenarnya dapat digantikan dengan sebuah pernyataan berupa

```
y = x;
```

Seandainya pada program di atas tidak terdapat pernyataan

```
px = &x;
```

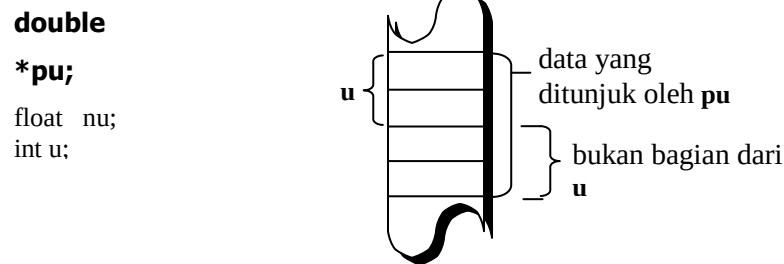
namun terdapat pernyataan

```
y = *px;
```

maka **y** tidaklah berisi nilai **x**, sebab **px** belum diatur agar menunjuk ke variabel **x**. Hal seperti ini harap diperhatikan. Kalau program melibatkan pointer, dan pointer belum diinisialisasi, ada kemungkinan akan terjadi masalah yang dinamakan “bug” yang bisa mengakibatkan komputer tidak dapat dikendalikan (*hang*).

Selain itu tipe variabel pointer dan tipe data yang ditunjuk harus sejenis. Bila tidak sejenis maka akan terjadi hasil yang tidak diinginkan.

Untuk lebih jelasnya lihat gambar berikut.



Gambar 19.3 Ilustrasi kesalahan yang terjadi karena tipe tidak

Mengakses dan Mengubah isi Suatu Variabel Pointer

Contoh berikut memberikan gambaran tentang perubahan isi suatu variabel secara tak langsung (yaitu melalui pointer). Mula-mula **pd** dideklarasikan sebagai pointer yang menunjuk ke suatu data bertipe *float* dan **d** sebagai variabel bertipe *float*. Selanjutnya

```
d = 54.5;
```

digunakan untuk mengisi nilai 54,5 secara langsung ke variabel **d**. Adapun

```
pd = &d;
```

digunakan untuk memberikan alamat dari **d** ke **pd**. Dengan demikian **pd** menunjuk ke variabel **d**. Sedangkan pernyataan berikutnya

```
*pd = *pd + 10;      (atau: *pd += 10; )
```

merupakan instruksi untuk mengubah nilai variabel **d** secara tak langsung. Perintah di atas berarti “jumlahkan yang ditunjuk **pd** dengan 10 kemudian berikan ke yang ditunjuk oleh **pd**”, atau identik dengan pernyataan

```
d = d + 10;
```

Akan tetapi, seandainya tidak ada instruksi

```
pd = &d;
```

maka pernyataan

```
*pd = *pd + 10;
```

tidaklah sama dengan

```
d = d + 10;
```

C. PERCOBAAN

Untuk setiap program di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

```
1. main(){
    int y, x = 87;
    int *px;

    px = &x;
    y = *px;

    printf("Alamat x      = %p\n", &x);
    printf("Isi px        = %p\n", px);
    printf("Isi x          = %d\n", x);
    printf("Nilai yang ditunjuk oleh px = %d\n", *px);
    printf("Nilai y        = %d\n", y);
}

2. main(){
    float *pu, nu;
    double u = 1234.0;

    pu = &u;
    nu = *pu;

    printf("Alamat dari u = %p\n", &u);
    printf("Isi pu          = %p\n", pu);
    printf("Isi u            = %lf\n", u);
    printf("Nilai yang ditunjuk oleh pu = %f\n", *pu);
    printf("Nilai nu          = %f\n", nu);
}

3. main(){
    float d = 54.5f, *pd;

    printf("Isi d mula-mula = %g\n", d);

    pd = &d;
    *pd += 10;

    printf("Isi d sekarang  = %g\n", d);
}
```

```

4. main(){
    int z = 20, s = 30, *pz, *ps;

    pz = &z;
    ps = &s;
    *pz += *ps;
    printf("z = %d\n", z);
    printf("s = %d\n", s);
}

5. main(){
    char c = 'Q', *cp = &c;

    printf("%c %c\n", c, *cp);
    c = '/';
    printf("%c %c\n", c, *cp);
    *cp = '(';
    printf("%c %c\n", c, *cp);
}

6. main() {
    int x = 1, y = 2, *ip;

    ip = &x;
    y = *ip;
    *ip = 3;

    printf("x = %d, y = %d", x, y);
}

7. main(){
    int i1, i2, *p1, *p2;

    i1 = 9;
    p1 = &i1;
    i2 = *p1 / 2 - 2 * 3;
    p2 = p1;

    printf("i1=%d,i2=%d,*p1=%d,*p2=%d\n", i1, i2, *p1, *p2);
}

8. main() {
    int count = 10, *temp, sum = 7;

    temp = &count;
    *temp = 32;
    temp = &sum;
    *temp = count;
    sum = *temp * 4;
}

```

```

    printf("count=%d, *temp=%d, sum=%d\n", count,*temp, sum );
}

9. main(){
    int count = 13, sum = 9, *x, *y;

    x = &count;
    *x = 27;
    y = x;
    x = &sum;
    *x = count;
    sum = *x / 2 * 3;

    printf("count=%d, sum=%d, *x=%d, *y=%d\n", count,sum,*x,*y);
}

10. int r, q = 7;
    int go_crazy(int *, int *);

    main() {
        int *ptr1 = &q;
        int *ptr2 = &q;

        r = go_crazy(ptr1, ptr2);
        printf("q=%d, r=%d, *ptr1=%d, *ptr2=%d\n",q,r,*ptr1,*ptr2);

        ptr2 = &r;

        go_crazy(ptr2, ptr1);
        printf("q=%d, r=%d, *ptr1=%d, *ptr2=%d\n",q,r,*ptr1,*ptr2);
    }

    int go_crazy(int *p1, int *p2){
        int x = 5;

        r = 12;
        *p2 = *p1 * 2;
        p1 = &x;
        return *p1 * 3;
    }

```

D. LAPORAN RESMI

Kumpulkan listing program, ilustrasi alokasi memorinya beserta hasil eksekusinya.

Pointer 2

A. TUJUAN PEMBELAJARAN

1. Memahami tentang *Pointer to Array*
2. Memahami tentang *Pointer to String*

B. DASAR TEORI

Pointer dan Array (*pointer to array*)

Hubungan antara pointer dan array pada C sangatlah erat. Sebab sesungguhnya array secara internal akan diterjemahkan dalam bentuk pointer. Pembahasan berikut akan memberikan gambaran hubungan antara pointer dan array. Misalnya dideklarasikan di dalam suatu fungsi

```
static int tgl_lahir[3] = { 01, 09, 64 };
```

dan

```
int *ptgl;
```

Kemudian diberikan instruksi

```
ptgl = &tgl_lahir[0]; //pointer to array of integer
```

maka **ptgl** akan berisi alamat dari elemen array **tgl_lahir** yang berindeks nol. Instruksi di atas bisa juga ditulis menjadi

```
ptgl = tgl_lahir;
```

sebab nama array tanpa tanda kurung menyatakan alamat awal dari array. Sesudah penugasan seperti di atas,

```
*ptgl
```

dengan sendirinya menyatakan elemen pertama (berindeks sama dengan nol) dari array **tgl_lahir**.

Pointer dan String (*pointer to string*)

Contoh hubungan pointer dan string ditunjukkan pada program berikut.

```
//Program : ptr4.c
#include <stdio.h>

main() {
    //pkota menunjuk konstanta string "SEMARANG"
    char *pkota = "SEMARANG";

    printf("String yang ditunjuk oleh pkota = ");
    puts(pkota);           // printf("%s\n", pkota);
}
```

Contoh eksekusi :

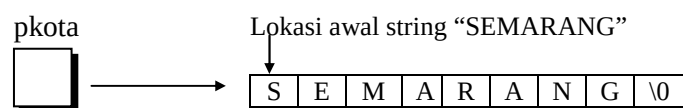
String yang ditunjuk oleh pkota = SEMARANG

Pada program di atas,

```
char *pkota = "SEMARANG";
```

akan menyebabkan kompiler

- mengalokasikan variabel **pkota** sebagai variabel pointer yang menunjuk ke obyek bertipe *char* dan menempatkan konstanta "SEMARANG" dalam suatu memori
- kemudian pointer **pkota** akan menunjuk ke lokasi string "SEMARANG".



Gambar 20.1 Pointer menunjuk data **string**

Pernyataan di atas menyerupai pernyataan

```
char kota[] = "SEMARANG";
```

tetapi sebenarnya kedua pernyataan inisialisasi di depan tidaklah tepat sama. Sebab **pkota** adalah pointer (menyatakan alamat) yang dengan mudah dapat diatur agar menunjuk ke string lain (bukan string "SEMARANG"), sedangkan **kota** adalah array (array menyatakan alamat yang konstan, tak dapat diubah).

Jadi, ada beberapa cara penulisan ekspresi variabel yang menghasilkan sebuah address memory, di antaranya adalah :

1. **Menuliskan ampersand di depan sebuah variabel 'normal'**

→ Misalnya a dideklarasikan sebagai sebuah variabel normal, maka &a akan menghasilkan adress memory dari variabel a tersebut

2. **Menuliskan nama dari sebuah variabel pointer**

→ Misalnya a dideklarasikan sebuah variabel sebagai `int *a`, maka a akan berisi sebuah address memory yang tersimpan pada variabel a (setelah ada sebuah address memory yang di-assign ke variabel a tersebut)

3. **Menuliskan nama sebuah variabel array tanpa indexnya**

→ Misalnya dideklarasikan

- `float nilai[5]`, maka penulisan `nilai` artinya sama dengan `&nilai[0]`
- `int b[3][4]`, maka `b[3]` adalah sama dengan `&b[3][0]` dan `b[0]` adalah sama dengan `&b[0][0]`.

Bisa disimpulkan bahwa sebuah variabel array yang dituliskan tanpa kurung siku indexnya adalah sama dengan address memory dari elemen pertama array tersebut

C. PERCOBAAN

Untuk setiap program pada no 1 – 8 di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

Untuk no. 9 buatlah programnya

1. Hubungan antara pointer dan array. Suatu nama array yang ditulis tanpa dengan indeksinya menunjukkan alamat elemen pertama dari array (versi 1).

```
#include <stdio.h>
main(){
    static int tgl_lahir[] = {16, 4, 1974};
    int *ptgl;
```

```

    ptgl = tgl_lahir;
    printf("Nilai yang ditunjuk oleh ptgl = %d\n", *ptgl);
    printf("Nilai dari tgl_lahir[0] = %d\n", tgl_lahir[0]);
}

```

2. Hubungan antara pointer dan array. Suatu nama array yang ditulis tanpa dengan indeksinya menunjukkan alamat elemen pertama dari array (versi 2).

```

#include <stdio.h>
main(){
    static int tgl_lahir[] = {16, 4, 1974};
    int *ptgl, i;

    ptgl = tgl_lahir;

    printf("Nilai yang ditunjuk oleh ptgl = %d\n", *ptgl);
    for (i=0; i<3; i++)
        printf("Nilai dari tgl_lahir[i] = %d\n", *(ptgl+i));
}

```

3. Hubungan antara pointer dan array. Suatu nama array yang ditulis tanpa dengan indeksinya menunjukkan alamat elemen pertama dari array (versi 3).

```

#include <stdio.h>
main(){
    static int tgl_lahir[] = {16, 4, 1974};
    int i;
    int *ptgl;

    ptgl = tgl_lahir;

    printf("Nilai yang ditunjuk oleh ptgl = %d\n", *ptgl);
    for (i=0; i<3; i++)
        printf("Nilai dari tgl_lahir[i] = %d\n", *ptgl++);
}

```

Analisislah dan jelaskan perbedaan antara aplikasi pada nomor 1, 2 dan 3

4. Menukarkan isi 2 string tanpa pemakaian pointer.

```

#include <stdio.h>
#include <string.h>

#define PANJANG 20

```

```

char nama1[PANJANG] = "AHMAD";
char nama2[PANJANG] = "RIFDA";

main(){
    char namax[PANJANG];

    puts("SEMULA : ");
    printf("nama1 --> %s\n", nama1);
    printf("nama2 --> %s\n", nama2);

    strcpy(namax, nama1);
    strcpy(nama1, nama2);
    strcpy(nama2, namax);

    puts("KINI : ");
    printf("nama1 --> %s\n", nama1);
    printf("nama2 --> %s\n", nama2);
}

```

5. Menukarkan isi 2 string dengan fasilitas pointer.

```

#include <stdio.h>

char *nama1 = "AHMAD";
char *nama2 = "RIFDA";

main(){
    char *namax;

    puts("SEMULA : ");
    printf("nama1 --> %s\n", nama1);
    printf("nama2 --> %s\n", nama2);

    namax = nama1;
    nama1 = nama2;
    nama2 = namax;

    puts("KINI : ");
    printf("nama1 --> %s\n", nama1);
    printf("nama2 --> %s\n", nama2);
}

```

Analisislah dan jelaskan perbedaan antara aplikasi pada nomor 4 dengan nomor 5

6. Penggunaan *variable index* pada *array* dan *variable index* pada *pointer*, untuk menunjuk suatu nilai data di dalam suatu *variable array*. Berikan analisis dan kesimpulan

```
main(){
    int nilai[10]={86,75,98,66,56,76,80,95,70,60};
    int index, *ip;

    printf("Mencetak menggunakan array\n");
    printf("Daftar nilai siswa\n\n");

    for(index=0; index<10; index++)
        printf("%3d",nilai[index]);

    puts("\n");
    printf("Mencetak menggunakan pointer dan index\n");
    printf("Daftar nilai siswa\n\n");

    for(index=0; index<10; index++)
        printf("%3d",*(nilai+index));

    puts("\n");
    printf("Mencetak menggunakan pointer\n");
    printf("Daftar nilai siswa\n\n");

    ip=nilai;
    for(index=0; index<10; index++)
        printf("%3d",*ip++);
}
```

7. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh potongan program di bawah ini

```
main()    {
    char *text_pointer = "Good morning!";

    for( ; *text_pointer != '\0'; ++text_pointer)
        printf("%c", *text_pointer);
}
```

8. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh potongan program di bawah ini

```
int array1[10], array2[10];
int *ip1, *ip2 = array2;
int *akhir = &array1[10];
```

```
for(ip1 = &array1[0]; ip1 < akhir; ip1++)  
    *ip2++ = *ip1;
```

9. Definisikan sebuah fungsi untuk membaca sebuah array (dengan tipe sembarang) menggunakan pointer. Buatlah program untuk membaca array tersebut dalam rangka mencari sebuah nilai tertentu dan laporkan hasilnya berhasil menemukan atau tidak

D. LAPORAN RESMI

1. Kumpulkan listing program, ilustrasi alokasi memorinya beserta hasil eksekusinya.
2. Untuk no 9 : kumpulkan listing, capture output dan analisisnya

Pointer 3

A. TUJUAN PEMBELAJARAN

1. Memahami tentang *Array of Pointer*
2. Memahami tentang *Pointer to Pointer*

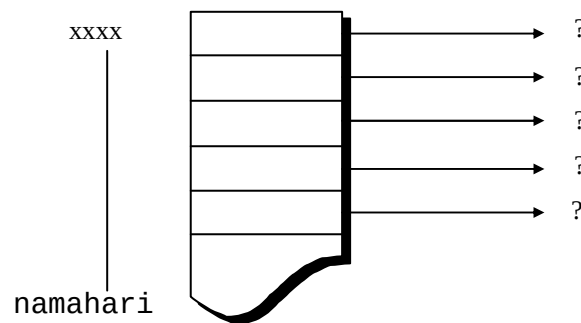
B. DASAR TEORI

Array dari Pointer (*Array of Pointer*)

- Suatu array bisa digunakan untuk menyimpan sejumlah pointer. Sebagai contoh:

```
char *namahari[10];
```

merupakan pernyataan untuk mendeklarasikan array pointer. Array **namahari** terdiri dari 10 elemen berupa pointer yang menunjuk ke data bertipe *char*. Ilustrasi penempatan dalam memori untuk array of pointer ini dapat dilihat pada gambar 26.1.



Gambar 26.1 *Array of Pointer*

- Array pointer bisa diinisialisasi sewaktu pendeklarasian. Sebagai contoh:

```
static char *namahari[] =  
{"Senin",  
 "Selasa",  
 "Rabu",  
 "Kamis",  
 "Jumat",  
 "Sabtu",  
 "Minggu"};
```

Pada contoh ini,

`namahari[0]` berisi alamat yang menunjuk ke string “Senin”.

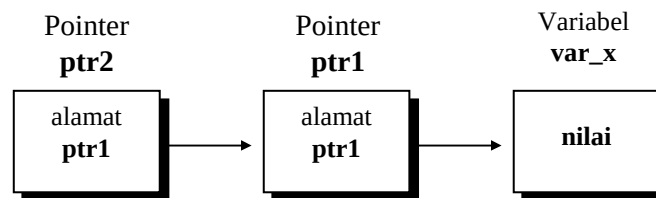
`namahari[1]` berisi alamat yang menunjuk ke string “Selasa”.

`namahari[2]` berisi alamat yang menunjuk ke string “Rabu”.

dan sebagainya.

Pointer menunjuk Pointer (*Pointer to Pointer*)

Suatu pointer bisa saja menunjuk ke pointer lain. Gambar 26.2 memberikan contoh mengenai pointer menunjuk pointer.



Gambar 26.2 Pointer yang menunjuk pointer

- Untuk membentuk rantai pointer seperti pada gambar di atas, pendeklarasian yang diperlukan berupa

```
int var_x;
int *ptr1;
int **ptr2;
```

Perhatikan pada deklarasi di depan:

- **var_x** adalah variabel bertipe `int`.
- **ptr1** adalah variabel pointer yang menunjuk ke data bertipe `int`.
- **ptr2** adalah variabel pointer yang menunjuk ke pointer `int`.
(itulah sebabnya deklarasinya berupa `int **ptr2;`)
- Agar **ptr1** menunjuk ke variabel **var_x**, perintah yang diperlukan berupa

```
ptr1 = &var_x;
```

- Sedangkan supaya **ptr2** menunjuk ke **ptr1**, instruksi yang diperlukan adalah
ptr2 = &ptr1;

C. PERCOBAAN

Untuk setiap program di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

1. Array of Pointer to char

```
main() {
    static char *days[] = {"Sun", "Mon", "Tues", "Wed", "Thu",
                           "Fri", "Sat"};
    int i;

    for( i = 0; i < 6; ++i )
        printf( "%s\n", days[i]);
}
```

2. Pointer yang menunjuk ke pointer yang lain.

```
main(){
    int a, *b, **c;

    a = 155;
    b = &a;
    c = &b;

    printf("Nilai a = %d atau %d atau %d\n", a, *b, **c);
    printf("b = %p = alamat a di memori\n", b);
    printf("c = %p = alamat b di memori\n", c);
    printf("alamat c di memori = %p\n", &c);
}
```

3. Pointer yang menunjuk ke pointer yang lain.

```
main(){
    int var_x = 273;
    int *ptr1;
    int **ptr2;

    ptr1 = &var_x;
    ptr2 = &ptr1;
```

```

printf("Nilai var_x = *ptr1 = %d\n", *ptr1);
printf("Nilai var_x = **ptr2 = %d\n\n", **ptr2);
printf("ptr1 = &var_x = %p\n", ptr1);
printf("ptr2 = &ptr1 = %p\n", ptr2);
printf("      &ptr2 = %p\n", &ptr2);
}

```

4. #include <stdio.h>

```

main(){
    int a, *b, **c;

    a = 1975;
    b = &a;
    c = &b;

    printf("Nilai a = %d atau %d atau %d\n", a, *b, **c);
    printf("b = %p = alamat a di memori\n", b);
    printf("c = %p = alamat b di memori\n", c);
    printf("alamat c di memori = %p\n", &c);
}

```

5. Untuk potongan program di bawah ini, gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses

```

int *i;
int j=10, k, m[]={2, 5};
int **l;

i = m;
i++;
*i = j;
j = *i;
i = &j;
k = *(&j);
l = &i;

```

6. Tentukan setiap statemen di bawah ini benar atau salah. Jika salah sertakan alasannya.

Deklarasi :

```

int a[5] = {2, 4, 8, 1, 23};
int c = 5;
int *ptr1 = &c;
int *ptr2 = a;

```

Statement	Benar (beri tanda X)	Salah (beri tanda X)
<code>a = c;</code>		
<code>*c = 6;</code>		
<code>a[2] = c;</code>		
<code>*ptr2 = c;</code>		
<code>&ptr1 = c;</code>		
<code>*(ptr2 + 1) = *(a + 3);</code>		
<code>c = *(ptr2 + 1);</code>		
<code>c = &ptr1;</code>		
<code>c = a[3] + 2;</code>		
<code>*(ptr2 + 2) = *ptr1;</code>		

D. LAPORAN RESMI

Kumpulkan listing program, ilustrasi alokasi memorinya beserta hasil eksekusinya.

Pointer 4

A. TUJUAN PEMBELAJARAN

1. Memahami tentang Pointer dalam Fungsi
2. Memahami tentang Pointer sebagai Parameter Fungsi
3. Memahami tentang Pointer sebagai Keluaran Fungsi

B. DASAR TEORI

Pointer dalam Fungsi

Pointer dan kaitannya dengan fungsi yang akan dibahas berikut meliputi :

- Pointer sebagai parameter fungsi
- Pointer sebagai keluaran fungsi

Pointer Sebagai Parameter Fungsi

- Penerapan pointer sebagai parameter yaitu jika diinginkan agar nilai suatu variabel internal dapat diubah oleh fungsi yang dipanggil.
- Sebagai contoh dapat dilihat pada fungsi berikut.

```
void naikkan_nilai (int *x, int *y){  
    *x = *x + 2;  
    *y = *y + 2;  
}
```

- Fungsi di atas dimaksudkan agar kalau dipanggil, variabel yang berkenaan dengan parameter aktual dapat diubah nilainya, masing-masing dinaikkan sebesar 2. Contoh pemanggilan :

```
    naikkan_nilai(&a, &b);
```

- Perhatikan, dalam hal ini variabel **a** dan **b** harus ditulis diawali operator alamat (**&**) yang berarti menyatakan alamat variabel, sebab parameter fungsi dalam pendefinisian berupa pointer.

Pointer Sebagai Keluaran Fungsi (*return value*)

- Suatu fungsi dapat dibuat agar keluarannya berupa pointer. Misalnya, suatu fungsi menghasilkan keluaran berupa pointer yang menunjuk ke string `nama_bulan`, seperti pada contoh berikut.

```
char *nama_bulan(int n) {  
    static char *bulan[12]=  
        {"Kode bulan salah", "Januari", "Februari", "Maret",  
         "April", "Mei", "Juni", "Juli", "Agustus",  
         "September", "Oktober", "Nopember", "Desember"};  
    return ( (n<1 || n>12) ? bulan[0] : bulan[n] );  
}
```

- Pada definisi fungsi di atas,

```
char *nama_bulan()
```

menyatakan bahwa keluaran fungsi **nama_bulan()** berupa pointer yang menunjuk ke obyek char (atau string).

- Dalam fungsi **nama_bulan()**, mula-mula array bernama **bulan** dideklarasikan dan sekaligus diinisialisasi agar menunjuk sejumlah string yang menyatakan nama bulan. Di bagian akhir fungsi, pernyataan

```
return ( (n<1 || n>12) ? bulan[0] : bulan[n] );
```

menyatakan bahwa hasil fungsi berupa pointer yang menunjuk ke

→ string “Kode bulan salah” (**bulan[0]**) jika masukan fungsi `n<1` atau `n>12`

→ **bulan[n]** untuk `n` yang terletak antara 1 sampai dengan 12.

C. PERCOBAAN

Untuk setiap program di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

1. Fungsi dengan argumen berupa pointer.

```
#include <stdio.h>
void naikan_nilai(int *x, int *y);
main() {
    int a = 3;
    int b = 7;

    printf("SEMULA : a = %d  b = %d\n", a, b);
    naikan_nilai(&a, &b);
    printf("KINI      : a = %d  b = %d\n", a, b);
}

void naikan_nilai(int *x, int *y){
    *x = *x + 2;
    *y = *y + 2;
}
```

2. Fungsi dengan keluaran berupa pointer yang menunjuk string.

```
#include <stdio.h>
char *nama_bulan(int n);

main(){
    int bl;

    printf("Bulan 1..12 : ");
    scanf("%d", &bl);
    printf("Bulan ke-%d adalah %s\n", bl, nama_bulan(bl));
}

char *nama_bulan(int n){
    static char *bulan[] = {
        "Ngawur",
        "Januari",
        "Februari",
        "Maret",
        "April",
        "Mei",
        "Juni",
        "Juli",
        "Agustus",
        "September",
        "Oktober",
        "November",
        "Desember"
    };
};
```

```

    return ((n<1||n>12) ? bulan[0] : bulan[n]);
}

```

3. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh program di bawah ini dan tampilkan hasil eksekusinya.

```

#include <stdio.h>

char strA[80] = "A string to be used for demonstration";
char strB[80];

main(){
    char *pA, *pB;

    puts(strA);
    pA = strA;
    puts(pA);
    pB = strB;
    putchar('\n');

    while(*pA != '\0')
        *pB++ = *pA++;

    *pB = '\0';
    puts(strB);
}

```

Proses apakah yang sebenarnya dilakukan pada program tersebut ?

4. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh program di bawah ini

```

#include <stdio.h>
char *my_strcpy(char *, char *);

main(){
    char strA[80]="A string to be used for demonstration";
    char strB[80];

    my_strcpy(strB, strA);
    puts(strB);
}

char *my_strcpy(char *destination, char *source) {
    char *p = destination;
    while (*source != '\0')
        *p++ = *source++;
}

```

```

        *p = '\0';
        return destination;
    }

```

5. Bandingkan fungsi `my_strcpy()` di atas dengan fungsi `my_strcpy()` di bawah ini. Berikan penjelasan terhadap perbedaan proses dari kedua fungsi tersebut

```

char *my_strcpy(char dest[], char source[]){
    int i = 0;
    while (source[i] != '\0')
    {
        dest[i] = source[i];
        i++;
    }
    dest[i] = '\0';
    return dest;
}

```

6. Buatlah sebuah program dengan mendefinisikan sebuah fungsi `rotasi()` yang menerima tiga parameter berupa variabel `a`, `b`, dan `c`. Fungsi ini melakukan rotasi sehingga nilai `a` berpindah ke `b`, `b` ke `c` dan nilai `c` ke `a` sekembalinya ke fungsi `main()`.

D. LAPORAN RESMI

Kumpulkan listing program, ilustrasi alokasi memorinya beserta hasil eksekusinya.